



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales



Unidad de Aprendizaje:

Inteligencia Artificial

Grupo: 6CV2

Práctica: 1 (DFS y BFS)

Integrantes:

Barros Martínez Luis Enrique
Bautista Ríos Alfredo
García Martínez Ángel Yahel

Profesor: Marco Antonio Castillo

Introducción

La presente práctica aborda nuestro acercamiento a los algoritmos de búsqueda no informados (DFS y BFS), estos consisten en la selección e implementación de estrategias de búsqueda de un estado solución a partir de un estado inicial sin introducir al algoritmo de solución conocimiento sobre el impacto de las transiciones en la exploración del espacio de estados.

Tras ser implementados dichos algoritmos se obtendrá su análisis respecto al tiempo que les toma llegar a el dato en búsqueda con respecto a los casos ingresados.

Desarrollo

Se realizará la programación del algoritmo de Búsqueda en anchura (BFS) y Búsqueda en profundidad (DFS) en esta ocasión en lenguaje C++

BFS

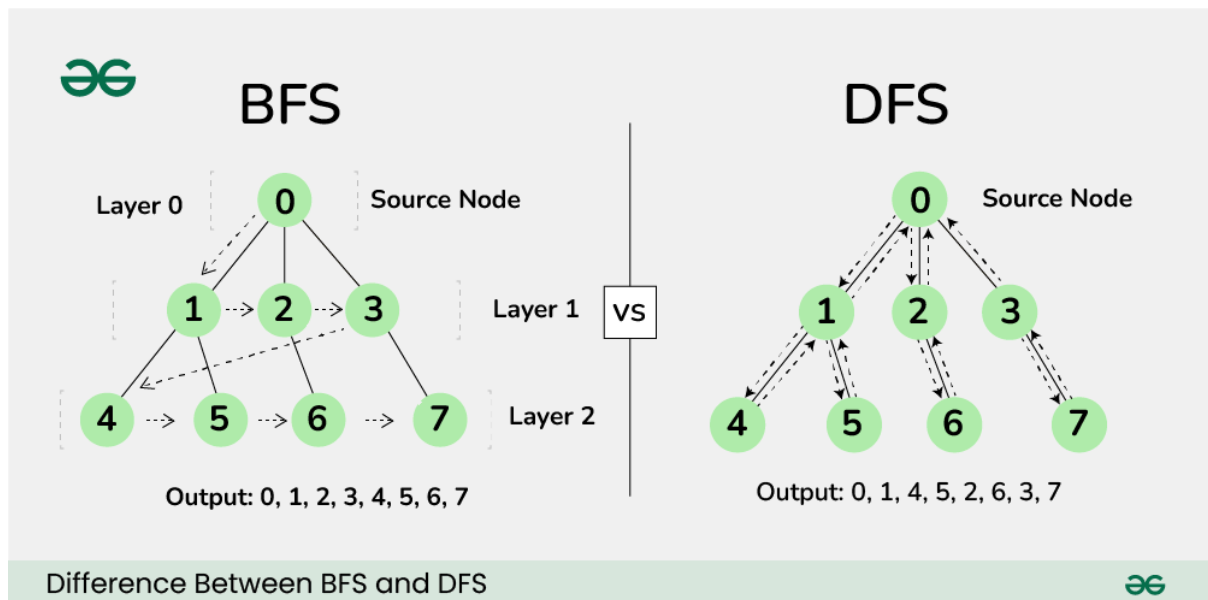
La búsqueda en anchura provee un algoritmo simple (basado en el recorrido de árboles por niveles) para encontrar un estado de solución desde un estado inicial. Al implementarse mediante el recorrido por niveles, se requiere una estructura cola (FIFO, first-in, first-out).

En inteligencia artificial, el algoritmo Breadth-First Search (BFS) es una herramienta esencial para explorar y navegar por diversos espacios de problemas. Al recorrer sistemáticamente estructuras de grafos o árboles, BFS resuelve tareas como la búsqueda de rutas, el enrutamiento de redes y la resolución de acertijos. Este artículo analiza los conceptos básicos de BFS, sus algoritmos y aplicaciones prácticas en IA.

DFS

Una búsqueda en profundidad (DFS) es un algoritmo de búsqueda para lo cual recorre los nodos de un grafo. Su funcionamiento consiste en ir expandiendo cada uno de los nodos que va localizando, de forma recurrente (desde el nodo padre hacia el nodo hijo). Cuando ya no quedan más nodos que visitar en dicho camino, regresa al nodo predecesor, de modo que repite el mismo proceso con cada uno de los vecinos del nodo. Cabe resaltar que si se encuentra el nodo antes de recorrer todos los nodos, concluye la búsqueda.

La búsqueda en profundidad surge como un algoritmo crucial en inteligencia artificial que ofrece tanto conocimientos teóricos como aplicaciones prácticas. Los DFS son una herramienta fundamental para atravesar espacios de estados complejos. A medida que la tecnología evoluciona, DFS sigue siendo un algoritmo fundamental para impulsar avances y ampliar los límites de la investigación y las aplicaciones de la IA.



Código:

El código mostrado a continuación es capaz de recibir cualquier grafo luego de indicarle (número de nodos, número de aristas, y las conexiones), se genera un número aleatorio dentro del rango del número de nodos, el cual será el nodo a encontrar realizando búsquedas con los algoritmos DFS y BFS, primero imprimirá en pantalla el número aleatorio a buscar, indicará su raíz, y el recorrido que haga según el algoritmo, concluyendo con que se mostrará al usuario el tiempo y memoria empleado en cada búsqueda. Nota: Se detendrá la búsqueda en el momento en el que se encuentre el nodo.

```
#include <bits/stdc++.h>
#include <chrono>
#include <sys/resource.h> // Para medir el uso de memoria.

using namespace std::chrono;
using namespace std;

const int N = 1e5; // Tamaño máximo permitido para el número de nodos.
vector<int>* g; // Puntero a un array de vectores que representará el grafo.
int vis[N]; // Array para llevar un registro de los nodos visitados.
bool encontrado; // Variable de control para saber si el nodo fue encontrado.

// Función para obtener el uso de memoria en kilobytes.
long getMemoryUsage() {
    struct rusage usage;
    getrusage(RUSAGE_SELF, &usage);
    return usage.ru_maxrss; // Retorna el máximo tamaño de memoria residente.
}

int ttime;
void dfs(int u, int x) {
    if (encontrado) return; // Si el nodo ya fue encontrado, detén la búsqueda.
    ttime++;
    cout << u << " "; // Imprime el nodo actual.
    if (u == x) {
        cout << "Me encontraste nodo" << u << " en el nivel: " << vis[u] << endl << " En " << ttime << " pasos,"; // Si se encuentra el nodo buscado, imprime un mensaje.
        encontrado = true; // Marca como encontrado.
        return; // Detiene la búsqueda.
    } else {
        cout << "\n";
    }
    for (auto it : g[u]) // Recorre los vecinos del nodo actual.
        if (!vis[it]) {
            vis[it] = vis[u] + 1; // Si un vecino no ha sido visitado, llama recursivamente a DFS.
            dfs(it, x);
        }
}

void bfs(int u, int x) {
    queue<int> q;
    q.push(u);
    vis[u] = 1;
    ttime = 0;
    while (!q.empty()) {
        ttime++;
        int v = q.front(); q.pop();
        cout << v << " "; // Imprime el nodo actual.
        if (v == x) {
```

```

        cout << "Me encontraste nodo" <<v<<" en el nivel: " <<vis[u]<<endl<<"
En " <<ttime<<" pasos,"; // Si encuentra el nodo buscado, imprime el mensaje.
        return; // Detén la búsqueda.
    } else {
        cout << "\n";
    }
    for (auto it : g[v]) { // Recorre los vecinos del nodo actual.
        if (!vis[it]) {
            q.push(it);
            vis[it] = vis[v]+1; // Marca los vecinos como visitados y los
añade a la cola.
        }
    }
}
}

int main() {
    int n, m;
    cin >> n >> m; // Lee el número de nodos y aristas del grafo.
    g = new vector<int>[n + 1]; // Inicializa la lista de adyacencia.

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b; // Lee las aristas.
        g[a].push_back(b); // Agrega la conexión en ambas direcciones (grafo no
dirigido).
        g[b].push_back(a);
    }

    srand(time(0)); // Inicializa el generador de números aleatorios.
    int numeroRaiz = 1; // Establece 1 como raíz.
    int numeroAleatorioEncontrar = rand() % n + 1; // Elige un nodo aleatorio a
buscar.
    cout << "Raiz: " <<numeroRaiz <<" Buscar: " <<numeroAleatorioEncontrar <<
endl;

    // DFS
    cout << "DFS" << "\n";
    for (int i = 0; i <= n; i++) vis[i] = 0; // Reinicia el array de nodos
visitados.
    encontrado = false; // Resetea la variable de control.
    auto start = high_resolution_clock::now(); // Captura el tiempo inicial.
    long memoriaInicialDFS = getMemoryUsage(); // Obtén la memoria inicial.
    vis[numeroRaiz]=1;
    ttime=0;
    dfs(numeroRaiz, numeroAleatorioEncontrar); // Llama a la DFS.
    auto end = high_resolution_clock::now(); // Captura el tiempo final.
    long memoriaFinalDFS = getMemoryUsage(); // Obtén la memoria final.
    auto duration = duration_cast<microseconds>(end - start); // Calcula la
duración.
    cout <<"en " << duration.count() << "ms"; // Imprime el tiempo de ejecución.
    cout << (memoriaFinalDFS - memoriaInicialDFS) * 1024 << " B\n"; // Imprime
la memoria utilizada en bytes.

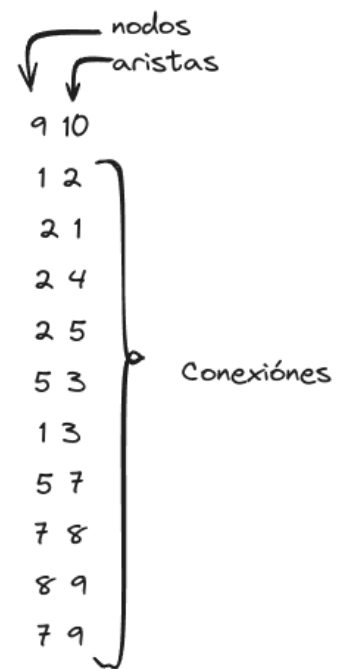
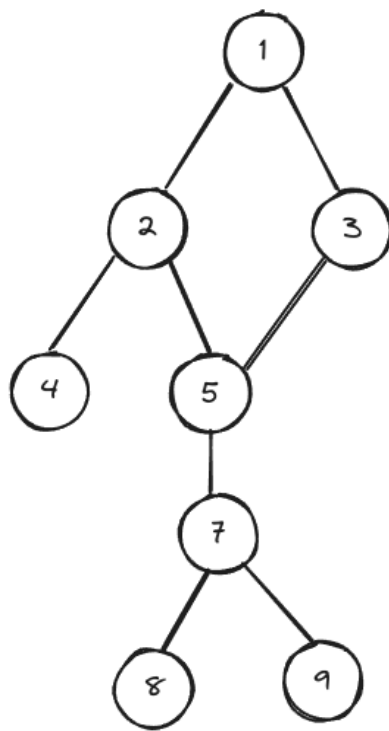
    // BFS
    cout << "BFS" << "\n";
    for (int i = 0; i <= n; i++) vis[i] = 0; // Reinicia el array de nodos
visitados.
    start = high_resolution_clock::now(); // Captura el tiempo inicial.
    long memoriaInicialBFS = getMemoryUsage(); // Obtén la memoria inicial.
    bfs(numeroRaiz, numeroAleatorioEncontrar); // Llama a la BFS.
    end = high_resolution_clock::now(); // Captura el tiempo final.
    long memoriaFinalBFS = getMemoryUsage(); // Obtén la memoria final.

```

```
        duration = duration_cast<microseconds>(end - start);    // Calcula la
duración.
        cout << "en " << duration.count() << "ms y ";    // Imprime el tiempo de
ejecución.
        cout << (memoriaFinalBFS - memoriaInicialBFS) * 1024 << " B\n";    // Imprime
la memoria utilizada en bytes.

        return 0;    // Indica que el programa terminó correctamente.
    }
```

Ejemplo Caso 1:



```

alfredo@archlinux ~/Documents/ESCOM-IS
C/semster 25-1/06 - Inteligencia Artific
ial/Practicas > ? main ± ./P1
9 10
1 2
2 1
2 4
2 5
5 3
1 3
5 7
7 8
8 9
7 9
Raiz: 1 Buscar: 9
DFS
1
2
4
5
3
7
8
9 Me encontraste nodo9 en el nivel: 6
En 8 pasos,en 26ms0 B
BFS
1
2
3
4
5
7
8
9 Me encontraste nodo9 en el nivel: 1
En 8 pasos,en 26ms y 0 B

```

```

alfredo@archlinux ~/Documents/ESCOM-IS
C/semster 25-1/06 - Inteligencia Artific
ial/Practicas > ? main ± ./P1
9 10
1 2
2 1
2 4
2 5
5 3
1 3
5 7
7 8
8 9
7 9
Raiz: 1 Buscar: 5
DFS
1
2
4
5 Me encontraste nodo5 en el nivel: 3
En 4 pasos,en 16ms0 B
BFS
1
2
3
4
5 Me encontraste nodo5 en el nivel: 1
En 5 pasos,en 17ms y 0 B

```

```

alfredo@archlinux ~/Documents/ESCOM-IS
C/semster 25-1/06 - Inteligencia Artific
ial/Practicas > ? main ± ./P1
9 10
1 2
2 1
2 4
2 5
5 3
1 3
5 7
7 8
8 9
7 9
Raiz: 1 Buscar: 6
DFS
1
2
4
5
3
7
8
9
en 42ms0 B
BFS
1
2
3
4
5
7
8
9
en 46ms y 0 B

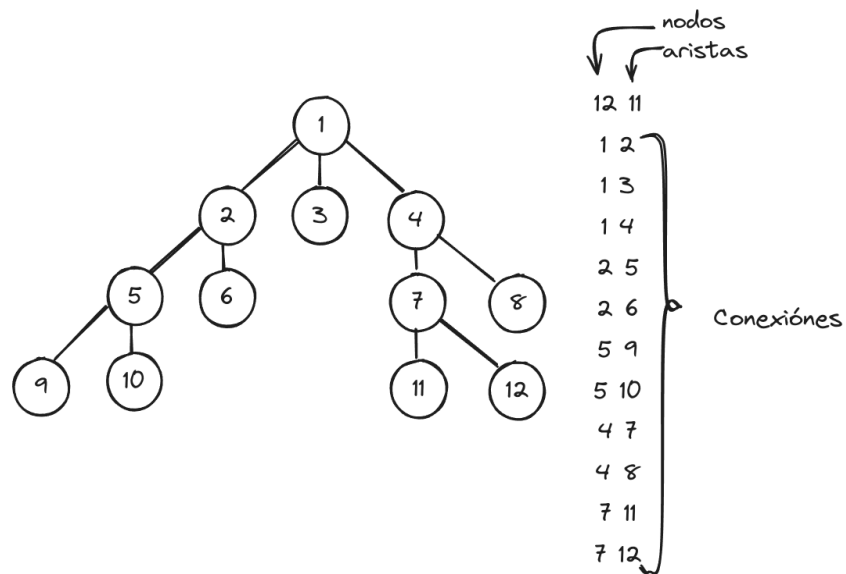
```

```

alfredo@archlinux ~/Documents/ESCOM-IS
C/semster 25-1/06 - Inteligencia Artific
ial/Practicas > ? main ± ./P1
9 10
1 2
2 1
2 4
2 5
5 3
1 3
5 7
7 8
8 9
7 9
Raiz: 1 Buscar: 2
DFS
1
2 Me encontraste nodo2 en el nivel: 2
En 2 pasos,en 13ms0 B
BFS
1
2 Me encontraste nodo2 en el nivel: 1
En 2 pasos,en 9ms y 0 B

```

Ejemplo Caso 2:



```
alfredo@archlinux ~/Documents/ESCOM-IS
C/semester 25-1/06 - Inteligencia Artific
ial/Practicas $ main ± ./P1
12 11
1 2
1 3
1 4
2 5
2 6
5 9
5 10
4 7
4 8
7 11
7 12
Raiz: 1 Buscar: 10
DFS
1
2
5
9
10 Me encontraste nodo10 en el nivel: 4
En 5 pasos,en 44ms0 B
BFS
1
2
3
4
5
6
7
8
9
10 Me encontraste nodo10 en el nivel: 1
En 10 pasos,en 67ms y 0 B
```

```
ial/Practicas $ main ± ./P1
12 11
1 2
1 3
1 4
2 5
2 6
5 9
5 10
4 7
4 8
7 11
7 12
Raiz: 1 Buscar: 12
DFS
1
2
5
9
10
6
3
4
7
11
12 Me encontraste nodo12 en el nivel: 4
En 11 pasos,en 58ms0 B
BFS
1
2
3
4
5
6
7
8
9
10
11
12 Me encontraste nodo12 en el nivel: 1
En 12 pasos,en 54ms y 0 B
```

```
alfredo@archlinux ~/Documents/ESCOM-IS
C/semester 25-1/06 - Inteligencia_Artific
ial/Practicas $ main ± ./P1
12 11
1 2
1 3
1 4
2 5
2 6
5 9
5 10
4 7
4 8
7 11
7 12
Raiz: 1 Buscar: 6
DFS
1
2
5
9
10
6 Me encontraste nodo6 en el nivel: 3
En 6 pasos,en 37ms0 B
BFS
1
2
3
4
5
6 Me encontraste nodo6 en el nivel: 1
En 6 pasos,en 37ms y 0 B

alfredo@archlinux ~/Documents/ESCOM-IS
C/semester 25-1/06 - Inteligencia_Artific
ial/Practicas $ main ± ./P1
12 11
1 2
1 3
1 4
2 5
2 6
5 9
5 10
4 7
4 8
7 11
7 12
Raiz: 1 Buscar: 9
DFS
1
2
5
9 Me encontraste nodo9 en el nivel: 4
En 4 pasos,en 28ms0 B
BFS
1
2
3
4
5
6
7
8
9 Me encontraste nodo9 en el nivel: 1
En 9 pasos,en 52ms y 0 B
```

Conclusión

Luego de las comparativas se ha podido observar, que el uso de tiempo y memoria usado en grafos con por lo menos una decena de nodos, arrojan una cantidad mínima de recursos usados, con esa cantidad es posible notar que la cantidad de milisegundos usados varía en proporción a en qué momento se detiene la búsqueda, tras investigar notamos que BFS consume más memoria que DFS, ya que almacena todos los nodos por nivel, mientras que DFS solo guarda el camino actual, lo que lo hace más eficiente en memoria. Ambos tienen una complejidad temporal de $O(V + E)$, pero BFS es más rápido para encontrar caminos más cortos en grafos no ponderados, ya que explora por niveles, mientras que DFS puede realizar más pasos al seguir rutas inútiles antes de encontrar la solución. BFS es ideal para hallar la solución más cercana al inicio, y DFS es preferible si la solución está más profunda o no importa la ruta óptima.



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales



Unidad de Aprendizaje:

Inteligencia Artificial

Grupo: 6CV2

Práctica

2 (A*)

Integrantes:

Barros Martínez Luis Enrique

Bautista Ríos Alfredo

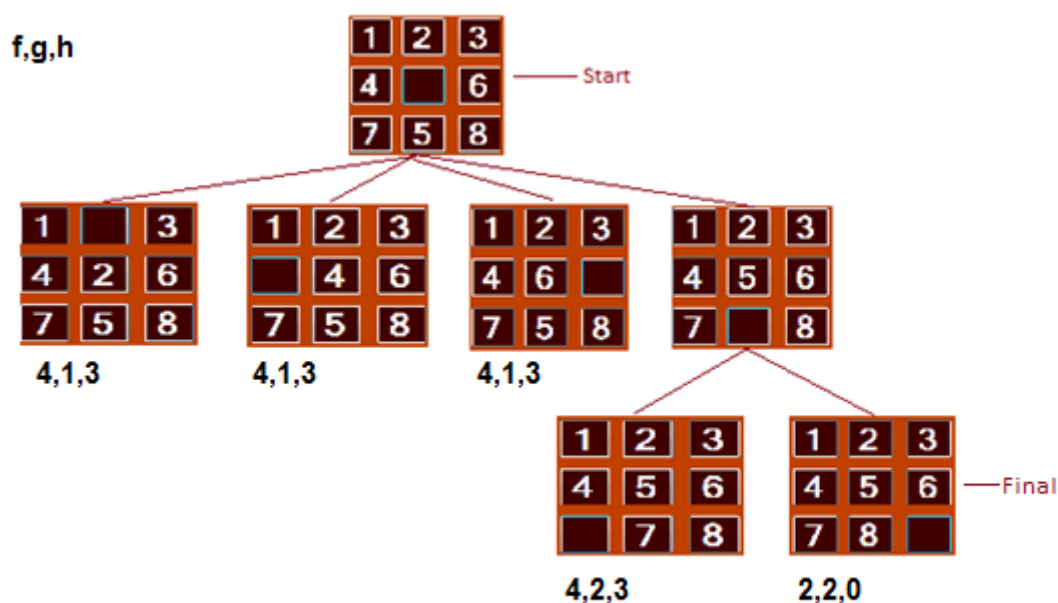
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El **algoritmo A*** es uno de los métodos más eficientes y populares para la búsqueda de rutas óptimas en grafos o espacios bidimensionales, como mapas o laberintos. Este algoritmo combina las ventajas de los algoritmos **Greedy** y **Dijkstra**, utilizando una función de evaluación $f(n)=g(n)+h(n)$, donde **g(n)** representa el costo acumulado desde el inicio hasta la celda actual, y **h(n)** es la heurística que estima el costo restante para alcanzar el objetivo. Su objetivo principal es encontrar el camino más corto y eficiente, evaluando múltiples alternativas para minimizar el costo total del recorrido.

A* Se aplica en problemas como la navegación de robots, videojuegos, y sistemas de GPS. Es especialmente útil cuando el espacio de búsqueda tiene restricciones como obstáculos, celdas bloqueadas o diferentes costos de movimiento, tal como se observa en esta práctica. El uso de listas **abierta** y **cerrada** permite organizar los nodos por explorar y los que ya se han evaluado, respectivamente. El algoritmo termina cuando el nodo objetivo ha sido encontrado o no hay más celdas accesibles por explorar.



Primer caso:

$A = \{A, O\}$
 $C = \{\}$

A	B	D	E	
•	←	←		
	10 70	20 60	30 70	
C	X	X	X	
10 70				
X				
X	X		X	X
	X		X	
	X	..	X	

①

$$A = \{A, O\}$$

$$C = \{\}$$

②

$$A = \{F, I, J, K\}$$

$$C = \{B, C, D, E, G, H\}$$

A	B	C	D	E	F
•	←	←	←	←	←
10	70	20	40	30	70
C	80				G
↑	X	X	X	X	114
10	70				44
X		J	H	I	
		108	108	114	
		68	40	58	50
X	X	K		X	X
		101			
		72	30		
	X		X		
	X	..	X		

①

$A = \{A, O\}$
 $C = \{\}$

②

$A = \{F, I, J, K\}$
 $C = \{B, C, D, E, G, H\}$

③

$A = \{F, I, J, K, L, M, O, N\}$
 $C = \{B, C, D, E, G, H, K\}$

A	B	C	D	E	F
80	70	60	50	40	30
10	20	30	40	50	60
70	80	90	100	110	120
130	140	150	160	170	180
190	200	210	220	230	240
250	260	270	280	290	300
310	320	330	340	350	360
370	380	390	400	410	420
430	440	450	460	470	480
490	500	510	520	530	540
550	560	570	580	590	600
610	620	630	640	650	660
670	680	690	700	710	720
730	740	750	760	770	780
790	800	810	820	830	840
850	860	870	880	890	900
910	920	930	940	950	960
970	980	990	1000	1010	1020
1030	1040	1050	1060	1070	1080
1090	1100	1110	1120	1130	1140
1150	1160	1170	1180	1190	1200
1210	1220	1230	1240	1250	1260
1270	1280	1290	1300	1310	1320
1330	1340	1350	1360	1370	1380
1390	1400	1410	1420	1430	1440
1450	1460	1470	1480	1490	1500
1510	1520	1530	1540	1550	1560
1570	1580	1590	1600	1610	1620
1630	1640	1650	1660	1670	1680
1690	1700	1710	1720	1730	1740
1750	1760	1770	1780	1790	1800
1810	1820	1830	1840	1850	1860
1870	1880	1890	1900	1910	1920
1930	1940	1950	1960	1970	1980
1990	2000	2010	2020	2030	2040
2050	2060	2070	2080	2090	2100
2110	2120	2130	2140	2150	2160
2170	2180	2190	2200	2210	2220
2230	2240	2250	2260	2270	2280
2290	2300	2310	2320	2330	2340
2350	2360	2370	2380	2390	2400
2410	2420	2430	2440	2450	2460
2470	2480	2490	2500	2510	2520
2530	2540	2550	2560	2570	2580
2590	2600	2610	2620	2630	2640
2650	2660	2670	2680	2690	2700
2710	2720	2730	2740	2750	2760
2770	2780	2790	2800	2810	2820
2830	2840	2850	2860	2870	2880
2890	2900	2910	2920	2930	2940
2950	2960	2970	2980	2990	3000
3010	3020	3030	3040	3050	3060
3070	3080	3090	3100	3110	3120
3130	3140	3150	3160	3170	3180
3190	3200	3210	3220	3230	3240
3250	3260	3270	3280	3290	3300
3310	3320	3330	3340	3350	3360
3370	3380	3390	3400	3410	3420
3430	3440	3450	3460	3470	3480
3490	3500	3510	3520	3530	3540
3550	3560	3570	3580	3590	3600
3610	3620	3630	3640	3650	3660
3670	3680	3690	3700	3710	3720
3730	3740	3750	3760	3770	3780
3790	3800	3810	3820	3830	3840
3850	3860	3870	3880	3890	3900
3910	3920	3930	3940	3950	3960
3970	3980	3990	4000	4010	4020
4030	4040	4050	4060	4070	4080
4090	4100	4110	4120	4130	4140
4150	4160	4170	4180	4190	4200
4210	4220	4230	4240	4250	4260
4270	4280	4290	4300	4310	4320
4330	4340	4350	4360	4370	4380
4390	4400	4410	4420	4430	4440
4450	4460	4470	4480	4490	4500
4510	4520	4530	4540	4550	4560
4570	4580	4590	4600	4610	4620
4630	4640	4650	4660	4670	4680
4690	4700	4710	4720	4730	4740
4750	4760	4770	4780	4790	4800
4810	4820	4830	4840	4850	4860
4870	4880	4890	4900	4910	4920
4930	4940	4950	4960	4970	4980
4990	5000	5010	5020	5030	5040
5050	5060	5070	5080	5090	5100
5110	5120	5130	5140	5150	5160
5170	5180	5190	5200	5210	5220
5230	5240	5250	5260	5270	5280
5290	5300	5310	5320	5330	5340
5350	5360	5370	5380	5390	5400
5410	5420	5430	5440	5450	5460
5470	5480	5490	5500	5510	5520
5530	5540	5550	5560	5570	5580
5590	5600	5610	5620	5630	5640
5650	5660	5670	5680	5690	5700
5710	5720	5730	5740	5750	5760
5770	5780	5790	5800	5810	5820
5830	5840	5850	5860	5870	5880
5890	5900	5910	5920	5930	5940
5950	5960	5970	5980	5990	6000
6010	6020	6030	6040	6050	6060
6070	6080	6090	6100	6110	6120
6130	6140	6150	6160	6170	6180
6190	6200	6210	6220	6230	6240
6250	6260	6270	6280	6290	6300
6310	6320	6330	6340	6350	6360
6370	6380	6390	6400	6410	6420
6430	6440	6450	6460	6470	6480
6490	6500	6510	6520	6530	6540
6550	6560	6570	6580	6590	6600
6610	6620	6630	6640	6650	6660
6670	6680	6690	6700	6710	6720
6730	6740	6750	6760	6770	6780
6790	6800	6810	6820	6830	6840
6850	6860	6870	6880	6890	6900
6910	6920	6930	6940	6950	6960
6970	6980	6990	7000	7010	7020
7030	7040	7050	7060	7070	7080
7090	7100	7110	7120	7130	7140
7150	7160	7170	7180	7190	7200
7210	7220	7230	7240	7250	7260
7270	7280	7290	7300	7310	7320
7330	7340	7350	7360	7370	7380
7390	7400	7410	7420	7430	7440
7450	7460	7470	7480	7490	7500
7510	7520	7530	7540	7550	7560
7570	7580	7590	7600	7610	7620
7630	7640	7650	7660	7670	7680
7690	7700	7710	7720	7730	7740
7750	7760	7770	7780	7790	7800
7810	7820	7830	7840	7850	7860
7870	7880	7890	7900	7910	7920
7930	7940	7950	7960	7970	7980
7990	8000	8010	8020	8030	8040
8050	8060	8070	8080	8090	8100
8110	8120	8130	8140	8150	8160
8170	8180	8190	8200	8210	8220
8230	8240	8250	8260	8270	8280
8290	8300	8310	8320	8330	8340
8350	8360	8370	8380	8390	8400
8410	8420	8430	8440	8450	8460
8470	8480	8490	8500	8510	8520
8530	8540	8550	8560	8570	8580
8590	8600	8610	8620	8630	8640
8650	8660	8670	8680	8690	8700
8710	8720	8730	8740	8750	8760
8770	8780	8790	8800	8810	8820
8830	8840	8850	8860	8870	8880
8890	8900	8910	8920	8930	8940
8950	8960	8970	8980	8990	9000
9010	9020	9030	9040	9050	9060
9070	9080	9090	9100	9110	9120
9130	9140	9150	9160	9170	9180
9190	9200	9210	9220	9230	9240
9250	9260	9270	9280	9290	9300
9310	9320	9330	9340	9350	9360
9370	9380	9390	9400	9410	9420
9430	9440	9450	9460	9470	9480
9490	9500	9510	9520	9530	9540
9550	9560	9570	9580	9590	9600
9610	9620	9630	9640	9650	9660
9670	9680	9690	9700	9710	9720
9730	9740	9750	9760	9770	9780
9790	9800	9810	9820	9830	9840
9850	9860	9870	9880	9890	9900
9910	9920	9930	9940	9950	9960
9970	9980	9990	10000	10010	10020
10030	10040	10050	10060	10070	10080
10090	10100	10110	10120	10130	10140
10150	10160	10170	10180	10190	10200
10210	10220	10230	10240	10250	10260
10270	10280	10290	10300	10310	10320
10330	10340	10350	10360	10370	10380
10390	10400	10410	10420	10430	10440
10450	10460	10470	10480	10490	10500
10510	10520	10530	10540	10550	10560
10570	10580	10590	10600	10610	10620
10630	10640	10650	10660	10670	10680
10690	10700	10710	10720	10730	10740
10750	10760	10770	10780	10790	10800
10810	10820	10830	10840	10850	10860
10870	10880	10890	10900	10910	10920
10930	10940	10950	10960	10970	10980
10990	11000	11010	11020	11030	11040
11050	11060	11070	11080	11090	11100
11110	11120	11130	11140	11150	11160
11170	11180	11190	11200	11210	11220
11230	11240	11250	11260	11270	11280
11290	11300	11310	11320	11330	11340
11350	11360	11370	11380	11390	11400
11410	11420	11430	11440	11450	11460
11470	11480	11490	11500	11510	11520
11530	11540	11550	11560	11570	11580
11590	11600	11610	11620	11630	11640
11650	11660	11670	11680	11690	11700
11710	11720	11730	11740	11750	11760
11770	11780	11790	11800	11810	11820
11830	11840	11850	11		

①

$$A = \{A, O\}$$

$$C = \{\}$$

②

$$A = \{F, I, J, K\}$$

$$C = \{B, C, D, E, G, H\}$$

③

$$A = \{F, I, J, K, L, M, O, N\}$$

$$C = \{B, C, D, E, G, H, K\}$$

④

$$A = \{F, I, J, L, M, O\}$$

$$C = \{B, C, D, E, G, H, K, N, O, P\}$$

A	D	D	E	F
80	80	80	100	120
10	70	20	60	30
40	80	70	40	80
C				G
80				114
10	70			44
	L	J	H	I
	136	108	108	114
	86	50	68	40
			58	50
		K		
		102		
		72	30	
	M	N	O	
	116	102	46	
	86	30	86	30
		0	102	
		92	10	
		P		
		102	0	

Segundo Caso

① $A = \{A, B\}$
 $C = \{\}$

	X		X	X		
				X		
	X	X			X	
	..	X				
X	X	X	X	X		
.	A	30				
	10	20				
B	40					
10	30	X				

① $A = \{A, B\}$

$C = \{\}$

② $A = \{H, I, J\}$

$C = \{B, A, C, E, D, F, G\}$

	X		X	X		
				X		
	X	X			X	
	..	X				
X	X	X	X	X	I 104 54 50	
.	A 30 10 20	B 50 20 30	D 70 30 40	F 90 40 50	H 100 50 60	
B 40 10 30	X	C 64 24 40	E 84 34 50	G 104 44 60	J 124 54 70	

① $A = \{A, B\}$
 $C = \{\}$

② $A = \{H, I, J\}$
 $C = \{B, A, C, E, D, F, G\}$

③ $A = \{P, Q, M, L, N, J\}$
 $C = \{B, A, C, E, D, F, G, I, N\}$

	X		X	X		
				X		
	X	X	P 112 82 30 9 98	O 118 78 40 W 98	X	
	..	X	78 20	68 30	N 104 64 40	L 118 68 50
X	X	X	X	X	I 104 54 50	K 124 64 60
.	A 30 10 20	B 50 20 30	D 70 30 40	F 90 40 50	H 100 50 60	J 138 68 70
B 40 10 30	X	C 64 24 40	E 84 34 50	G 104 44 60	J 124 54 70	

$$\textcircled{1} \quad A = \{A, B\}$$

$$C = \{\}$$

$$\textcircled{4} \quad A = \{Y, Z, W, U, AB\}$$

$$C = \{B, A, C, E, D, F, G, I, M, Q, O, L, K, T, N, AA\}$$

$$\textcircled{2} \quad A = \{H, I, J\}$$

$$C = \{B, A, C, E, D, F, G\}$$

$$\textcircled{3} \quad A = \{Q, P, O, M, L, K, N, J\}$$

$$C = \{B, A, C, E, D, F, G, I, N\}$$

	X	U 146 ↓	X	X		
W 146 →	V 126 →	S 126 ↘	R 132 ↓		Y 152 ↘	Z 158 ↓
116 30 X	106 20	96 30	92 40		92 60	88 70
140 ↗	X	X	P 112 116 ↘	O 118 138 ↘	X	T 138 ↓
120 20 AB	140 ↑	AC 134 ↘	Q 98 →	N 98 ↘	H 104 ↓	L 118 ↗
130 10	134 0		78 20	68 30	64 40	68 50
X	X	X	X	X	I 104 ↘	K 124 ↑
					54 50	78 60
A 30 →	B 50 ←	D 70 ←	F 90 ←	H 100 ←	J 138 158 ↑	
10 20	20 30	30 40	40 50	50 60	60 68	70
B 40 ↑	X	C 64 ↘	E 84 ↘	G 104 ↘	J 124 ↘	AA 158 ↑
10 30		24 40	34 50	44 60	54 70	78 80

Conclusión

Durante la realización manual del algoritmo A*, se pudo apreciar cómo este método explora las celdas más prometedoras en cada paso, priorizando tanto la distancia recorrida como la distancia estimada hacia la meta. A lo largo del recorrido, se construyeron las **listas abierta y cerrada**, evaluando las celdas vecinas y descartando aquellas que ya no ofrecían un camino más eficiente. Además, se consideraron los costos variables entre movimientos horizontales, verticales y diagonales, los cuales agregaron complejidad al análisis del camino más corto.

El proceso permitió entender mejor las decisiones que toma A* al elegir el siguiente nodo a expandir, y cómo la heurística influye directamente en su rendimiento. Esta actividad también mostró la importancia de organizar la información correctamente, ya que un pequeño error en la lista abierta o cerrada podría llevar a una ruta no tan óptima.